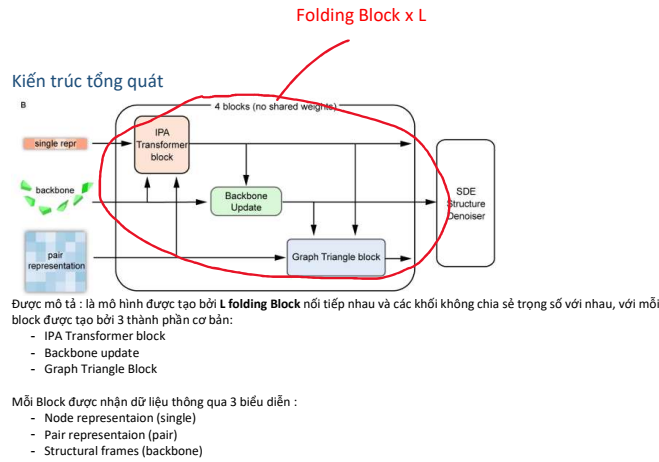




IPA-Transformer block

Đây là block được kết hợp ở IPA (Invariant Point Attention) và Transformer

Input : 3 loại representation

Output : biểu diễn node (s_{i+1}) được cập nhật

IPA (Invariant Point Attention) (Trong AlphaFold2)

Đây là method lấy cảm hứng từ AlphaFold2.

là một cơ chế attention đặc biệt được thiết kế để làm việc với thông tin không gian 3D của các khung protein. Nó đảm bảo rằng các phép toán attention không bị ảnh hưởng bởi các phép xoay và dịch chuyển toàn cục của cấu trúc protein

IPA kết hợp 3 biểu diễn đầu vào để tính toán trọng số attention và dùng nó để gán vào output.

Cụ thể, với mỗi residue nằm trong không gian riêng (mà ta đã tạo được thông qua trục chuẩn ở trên) sẽ có các Q, K, V riêng biệt. Sau đó, các điểm này được chiếu vào khung toàn cục của protein (Khung toàn cục được xác định bởi backbone frame của protein) và tương tác với nhau. Sau đó, chúng được chiếu lại vào không gian cục bộ để tính toán các trọng số attention cho mỗi residue. Điều này giúp duy trì thông tin cục bộ của mỗi residue mà vẫn đảm bảo các tính toán liên quan đến mối quan hệ toàn cục. Từ những thông tin có được, IPA cập nhật cấu trúc với mỗi vòng lặp mà không làm thay đổi vị trí 3D.

1 Cách để hiểu, IPA được dùng trong không gian 3D, đánh trọng số cho mỗi residue và thực hiện cập nhật node representation với mỗi vòng lặp.

Transformer

Sử dụng transformer tiêu chuẩn

IPA-Transformer block

Sử dụng hoàn toàn IPA từ AlphaFold2 và Transformer tiêu chuẩn trong paper.

Công dụng : Cập nhật thông tin nút

Kiến trúc của block:

$$s_\ell = \text{LayerNorm}(\text{IPA}(s_\ell, z_\ell, T_\ell) + s_\ell)$$

$$s_{in} = \text{concat}(s_\ell, \text{Linear}(s_0))$$

$$s_\ell = \text{Linear}(\text{Transformer}(s_{in})) + s_\ell$$

$$s_{\ell+1} = \text{MLP}(s_\ell)$$

Chi tiết mục này : https://scholar.google.com/scholar?url=https://arxiv.org/pdf/2302.02277&hl=en&sa=T&oi=gsr-r-gga&ct=res&cd=0&d=10606485028817870647&ei=3XaP29HNC6Oh6rQP5ePXuAc&scisig=AFWwaeaMB1naQbBEI-Y8cB-MUu_t

3XaP29HNC6Oh6rQP5ePXuAc&scisig=AFWwaeaMB1naQbBEI-Y8cB-MUu_t

Backbone update layer

Input :

- Biểu diễn node (residue) được cập nhật từ IPA-Transformer block (s_{i+1})

- Khung cấu trúc (backbone) từ bước trước đó (T_i)

Output :

- Khung cấu trúc được cập nhật (T_{i+1})

Do bản chất của cấu trúc 3D backbone là do sự tương tác giữa các residue, nên sau khi có cấu trúc mới của các residue, backbone được cập nhật. Layer này chịu trách nhiệm dự đoán các bản cập nhật về tịnh tiến và xoay của mỗi residue. Nó sử dụng một lớp tuyến tính để dự đoán các thay đổi trong dịch chuyển và xoay của khung cấu trúc dựa trên biểu diễn chuỗi đã được cập nhật.

Biểu diễn toán học :

$$b_i, c_i, d_i, \hat{x}_i^{\text{update}} = \text{Linear}(s_{\ell+1}) \quad (1)$$

$$(a_i, b_i, c_i, d_i) = (1, b_i, c_i, d_i) / \sqrt{1 + b_i^2 + c_i^2 + d_i^2} \quad (2)$$

$$R_i^{\text{update}} = \begin{pmatrix} a_i^2 + b_i^2 - c_i^2 - d_i^2 & 2b_i c_i - 2a_i d_i & 2b_i d_i + 2a_i c_i \\ 2b_i c_i + 2a_i d_i & a_i^2 - b_i^2 + c_i^2 - d_i^2 & 2c_i d_i - 2a_i b_i \\ 2b_i d_i - 2a_i c_i & 2c_i d_i + 2a_i b_i & a_i^2 - b_i^2 - c_i^2 + d_i^2 \end{pmatrix} \quad (3)$$

$$T_i^{\text{update}} = (R_i^{\text{update}}, \hat{x}_i^{\text{update}})$$

$$T_{\ell+1} = T_\ell \cdot T_i^{\text{update}} \quad 4$$

Giải thích phương trình 1 :

- Mô hình sử dụng 1 lớp linear để từ một biểu diễn residue (s_{i+1}) tạo ra các giá trị bao gồm b_i, c_i, d_i là các quaternion và vector tịnh tiến x_i .

Note : Quaternion là một công cụ toán học để biểu diễn phép quay trong không gian 3 chiều. Một quaternion thường được biểu diễn dưới dạng : $q = a + bi + cj + dk$

trong đó :

o a, b, c, d là các số thực

o i, j, k là các đơn vị ảo thỏa mãn tính chất của số ảo

Việc biểu diễn phép quay có nhiều phương pháp, có thể dùng công thức sau

$$q = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(xi + yj + zk)$$

Tuy nhiên, trong bài toán này tác giả dùng ma trận quay, vì vậy nên phải có phương trình 2

Giải thích phương trình 2 : Để biểu diễn được phép quay, quaternion phải được coi là hợp lệ (tức là phải chuẩn hóa). Vì vậy phương trình này là để chuẩn hóa quaternion, và đó cũng là cách mà a được sinh ra. Quaternion hợp lệ là khi : $a^2 + b^2 + c^2 + d^2 = 1$

Giải thích phương trình 3 : Ma trận quay được tính theo công thức

Giải thích các phương trình 4 : Cập nhật backbone. Hệ quy chiếu của mỗi residue được cập nhật bằng cách áp dụng ma trận quay mới và vector tịnh tiến mới lên backbone cũ (công thức ở trên). Sau đó cập nhật thành hệ quy chiếu mới với công thức ở dưới

The graph triangle block

Input :

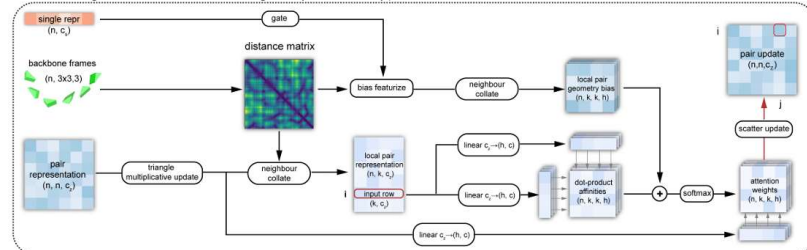
- Biểu diễn residue đã được cập nhật s_{i+1}
- Biểu diễn backbone đã được cập nhật T_{i+1}
- Biểu diễn cạnh trước đó z_i

Output :

- Biểu diễn cạnh được cập nhật z_{i+1}

Công dụng : cập nhật thông tin của biểu diễn cạnh

Đây là trái tim của kiến trúc, nơi làm nên sự khác biệt với các mô hình khác (Our primary emphasis is on elucidating the graph triangle block, which is the source of significant enhancements in designability and efficiency.)



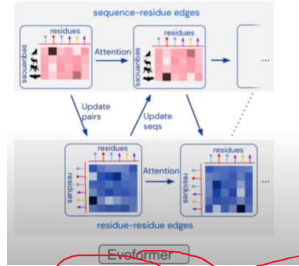
Block này sử dụng graph-based attention mechanism để xử lý edge representation

Mô hình bao gồm các folding block độc lập và không có sự chia sẻ trọng số giữa chúng.

Trong graph triangle block, tác giả đã tinh chỉnh lại triangle attention từ AlphaFold2. Triangle attention là trái tim của Evoformer module, cũng là trái tim của AlphaFold2. Tuy nhiên với Triangle attention, tác giả bắt gặp 2 vấn đề sau :

Độ phức tạp quá cao ($O(N^3)$)

Thiếu thông tin về backbone trong suốt quá trình huấn luyện, khi mà chỉ có thông tin về MSA và pairing correlation được đưa vào trong toàn bộ quá trình train module đó.



Để giải quyết vấn đề này, Graph triangle block được nhóm tác giả thiết kế ra.

(A) Block này áp dụng **triangle attention** với biểu diễn cạnh với độ phức tạp chỉ là $O(N.K^2)$, trong đó N là số lượng residue trong protein và K là K láng giềng gần nhất (nó được xem xét riêng cho mỗi residue) với khoảng cách tính toán bằng khoảng cách giữa các C-alpha. Lí do NK^2 là do mô hình xem xét K láng giềng cho mỗi residue, sau đó sử dụng N.K cạnh trong tập N^2 cạnh để tính toán attention.

(B) Để kết hợp với thông tin về không gian 3D, tác giả tính toán khoảng cách giữa các nguyên tử của **cạnh thứ 3**, khoảng cách này sau đó được dùng để tính RBF, RBF là một hàm có giá trị giảm dần khi khoảng cách đầu vào tăng. Các giá trị này được xem như là bias cấu trúc cho quá trình tính toán attention. Bias cấu trúc được điều khiển bởi 1 gate và 1 feedforward network, nó cho phép điều chỉnh cường độ của bias và khả năng kết hợp với 2 track còn lại.

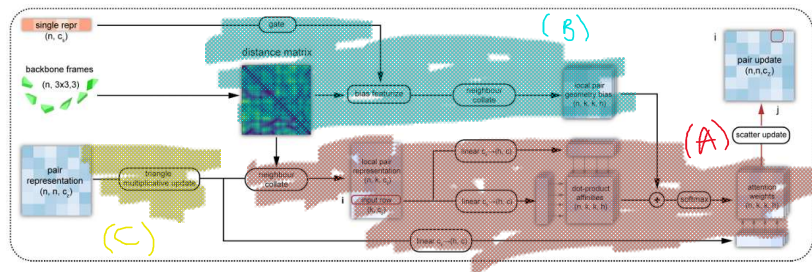
(C) Trước khi attention được thực hiện, tác giả sử dụng **triangle multiplicative update** để cập nhật toàn bộ biểu diễn cạnh ban đầu, lấy cảm hứng từ Evoformer của AlphaFold2.

Các lợi ích của việc sử dụng triangle update được chỉ ra trong RFDiffusion:

- tăng đáng kể khả năng của mô hình trong việc mô hình hóa quá trình diffusion backbone
- Giảm yêu cầu về bộ nhớ (so với attention)
- Kiến trúc phù hợp với nhiều task (không rõ cái này lắm ???)

Sau khi triangle multiplicative update hoàn thành, đến lượt của triangle attention để tiếp tục cập nhật trọng số của cạnh.

Từ những giải thích trên, em/minh xin được giải thích lại block này



Input into block :

- Single representation đã được cập nhật từ IPA
- Backbone representation đã được cập nhật từ Backbone update layer
- Pair representation

Các block con bao gồm :

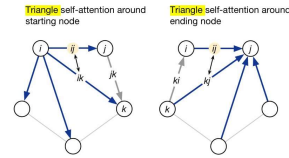
- Triangle multiplicative update: Biểu diễn cạnh ban đầu được cập nhật bằng cách kết hợp thông tin từ các mối quan hệ tam giác giữa các residue.
- Neighbour collate: Đối với mỗi residue, mô hình xác định K residue lân cận nhất dựa trên khoảng cách không gian.
- Local pair representation: Tạo biểu diễn cho các cặp residue lân cận.
- Distance matrix & Bias featurize: Tính toán ma trận khoảng cách giữa các residue và tạo các đặc trưng bias từ ma trận này.
- Local pair geometry bias: Kết hợp bias cấu trúc với biểu diễn cạnh cục bộ.
- Gate: Sử dụng biểu diễn chuỗi để điều chỉnh bias cấu trúc.
- Attention mechanism: Tính toán attention giữa các cạnh lân cận, sử dụng các thành phần sau:
 - Linear: Các lớp linear để biến đổi biểu diễn cạnh.
 - Dot-product affinities: Tính toán độ tương đồng giữa các cạnh.
 - Softmax: Chuẩn hóa các giá trị attention.
 - Attention weights: Trọng số attention cho mỗi cạnh.
- Pair update: Cập nhật biểu diễn cạnh dựa trên kết quả attention.
- Scatter update: Kết hợp các cập nhật từ attention cục bộ để tạo ra biểu diễn cạnh cuối cùng.

Note :

Triangle attention là biến thể của attention truyền thống, được thiết kế để khai thác thông tin từ 1 triangle. Nó được áp dụng cho đồ thị biểu diễn protein, với mỗi đỉnh là 1 residue và mỗi cạnh là mối quan hệ giữa chúng. Thay vì chỉ xem xét từng cặp thực thể, triangle attention đo lường mức độ ảnh hưởng lẫn nhau giữa ba cạnh trong một tam giác được tạo bởi ba residue, cụ thể là 2 cạnh lân cận còn lại.

Để triển khai triangle attention lên biểu diễn cạnh :

- Mô hình xác định tất cả các tam giác trong đồ thị (hoặc trong tập N.K)
- Tính toán attention
- Cập nhật biểu diễn cạnh



Đây là hình ảnh về triangle attention trong alphafold2 sau khi được update. Cụ thể, sau khi cập nhật trọng số cho cạnh ij như trên, ta tiếp tục cập nhật trọng số với triangle (các cạnh thứ 3 là logit bias nói trên) :

1. Cạnh ij sẽ cập nhật trọng số của nó dựa trên thông điệp đi ra từ nút i
2. Cạnh ij sẽ cập nhật trọng số của nó dựa trên thông điệp nhận được từ nút j

Note (cre: AlphaFold2)

Một cạnh trong tam giác đồ thị có thể được coi là "missing edge" nếu nó không trực tiếp xuất hiện trong bước tính toán hiện tại.

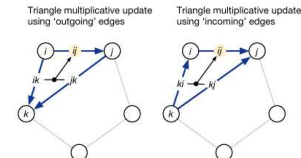
Để giải quyết vấn đề này, AlphaFold2 thêm một logit bias vào mô hình attention theo trục nhằm đưa thông tin từ hai cạnh còn lại để cập nhật cạnh bị thiếu này. Điều này cải thiện khả năng mô hình sử dụng thông tin toàn cục trong biểu diễn cặp

Note :

Triangle Multiplication update, được lấy ý tưởng từ AlphaFold2, sử dụng mối quan hệ giữa ba amino acid trong cấu trúc protein để cập nhật thông tin cho các cạnh trong đồ thị biểu diễn, tức là mỗi cạnh trong tam giác được cập nhật dựa trên thông tin từ hai cạnh còn lại.

(tác giả của alphafold2 nói rằng việc sử dụng đơn thuần Triangle Multiplicative update hoặc attention đã cho kết quả khá chính xác, tuy nhiên việc kết hợp cả 2 luôn cho ra kết quả tốt nhất)

Trong phần này không có 1 chút attention nào



2 kỹ thuật cập nhật dựa trên 'outgoing' và 'incoming' edge được trình bày trong AlphaFold2. Cụ thể hơn ta có thể thấy :

1. Trọng số của cạnh ij được cập nhật dựa trên các cạnh đi ra từ cái node của ij
2. Trọng số của ij được cập nhật dựa trên các cạnh đi vào ij

Architecture superiority

- Giải quyết được vấn đề bị phụ thuộc vào pretrained model của RFDiffusion
- Giải quyết được độ phức tạp $O(n^3)$ của AlphaFold2
- Với kiến trúc này, model có thể xử lý protein có 1024 residue so với 384 của AlphaFold2 và Rfdiffusion